

Wo entsteht Sentiment in Large Language Models?

*Eine mechanistische Analyse der Sentimentverarbeitung in
Pythia-410M*

AI Frameworks & Tools

Sommersemester 2026

Betreuer

Dr. Sigurd Schacht

Autoren

Alireza Roozitalab

Daniel Durst-Claus

Islam Abdalla

19. Juni 2026

1 Einleitung

Künstliche Intelligenz (KI) hat sich zu einem zentralen Forschungs- und Anwendungsgebiet der Informatik entwickelt, und die jüngsten Fortschritte in der Sprachverarbeitung beruhen vor allem auf der Transformer-Architektur und großen Sprachmodellen [1], [2]. Large Language Model (LLM) erreichen bei vielen sprachbezogenen Aufgaben hohe Leistung, ihr internes Verhalten bleibt jedoch häufig undurchsichtig. Die Mechanistische Interpretierbarkeit setzt genau hier an. Statt nur zu beobachten, was ein Modell ausgibt, untersucht sie, wie und warum eine Ausgabe im Inneren des Modells entsteht [3], [4].

Als Fallstudie dient in dieser Arbeit die Sentimentanalyse. Sentiment lässt sich gut mechanistisch untersuchen, weil es sich an einzelnen sentimenttragenden Wörtern festmachen lässt und sich positive und negative Beispiele mit nahezu identischem Kontext gegenüberstellen lassen. Untersucht wird das offene Modell Pythia-410M von EleutherAI [5], ausschließlich mit PyTorch und HuggingFace, ohne Spezialbibliotheken.

1.1 Hypothesen

Jede Hypothese ist mit einem konkreten Kriterium versehen, das angibt, wann sie als bestätigt gilt.

H1 (klare Stimmungswörter)

Eindeutig polare Wörter wie „wonderful“ oder „delicious“ werden früh und direkt am Token verarbeitet. Bestätigt, wenn die Polarität bereits in frühen bis mittleren Schichten ablesbar ist *und* ein Eingriff am Stimmungswort die Vorhersage wiederherstellt.

H2 (Ironie und Sarkasmus)

Das Modell erkennt keine Ironie, sondern verarbeitet nur die wörtliche Oberflächen-Polarität. Bestätigt, wenn das Modell durchgehend der wörtlichen Lesart folgt und die gemeinte ironische Polarität an keiner Schicht linear ablesbar ist.

H3 (Lokalisierbarkeit)

Die kausal verantwortliche Verarbeitung lässt sich auf eine bestimmte Schicht und Token-Position eingrenzen, statt über das gesamte Netz verteilt zu sein. Bestätigt, wenn eine scharf begrenzte Stelle das korrekte Verhalten nahezu vollständig wiederherstellt.

2 Grundlagen, Modell und Daten

2.1 Mechanistische Interpretierbarkeit

Die meiste Forschung zu Sprachmodellen beschränkt sich auf die Beziehung zwischen Eingabe und Ausgabe. Man beobachtet, was ein Modell tut, ohne zu wissen, wie es dazu kommt. Die Mechanistische Interpretierbarkeit geht einen Schritt weiter und öffnet die sprichwört-

liche Black Box. Ein Bild, das den Unterschied gut greifbar macht, ist der zwischen dem Fahren eines Autos und dem Verstehen seines Motors. Erst das Verständnis der inneren Abläufe erlaubt es, Fehler vorherzusagen und gezielt einzugreifen [3].

2.2 Residual Stream und fünf Werkzeuge

Moderne Transformers bestehen aus einem Stapel von Schichten, die alle auf einen gemeinsamen Informationskanal zugreifen, den *Residual Stream*. Jede Schicht liest daraus, berechnet einen Beitrag und addiert ihn wieder hinzu, ohne den bisherigen Inhalt zu löschen [4]. Man kann sich das wie ein Notizblatt vorstellen, auf das jede Schicht etwas ergänzt, statt es zu überschreiben; am Ende wird aus diesem Notizblatt die Vorhersage abgelesen.

Die Arbeit nutzt darauf aufbauend fünf Werkzeuge, die von beschreibenden hin zu kausalen Aussagen führen. Die **Residual-Stream-Analyse** und das **lineare Probing** prüfen, ob Sentiment in den Aktivierungen vorhanden und linear dekodierbar ist, also ob schon eine einfache Trennlinie reicht, um positiv von negativ zu unterscheiden. Der **Logit Lens** projiziert Zwischenrepräsentationen einzelner Schichten in den Vokabularraum [6] und zeigt so, ab wann das Modell eine Information nutzt. Die **Attention-Analyse** zeigt, welche Köpfe auf welche Token zugreifen, also wohin Information *fließt*. Nur das **Activation Patching** weist nach, dass eine Komponente kausal *benötigt* wird, indem einzelne Aktivierungen gezielt ersetzt und die Wirkung auf die Vorhersage gemessen wird [7], [8]. Die ersten vier Werkzeuge liefern Korrelationen, nur das letzte liefert einen echten kausalen Nachweis.

Für die Sentiment-Bewertung wird durchgehend das Opinion-Lexikon von Hu und Liu verwendet, das positive und negative Wörter wie „good“/„bad“ oder „excellent“/„terrible“ enthält [9].

2.3 Das Modell Pythia-410M

Pythia-410M ist ein offenes, vollständig dokumentiertes Decoder-Only-Transformer-Modell (GPT-NeoX-Architektur) von EleutherAI mit rund 405 Millionen Parametern [5]. Diese verteilen sich auf vier Hauptkomponenten. Embedding- und Unembedding-Matrix tragen je rund 12,7 Prozent, die Attention-Mechanismus-Blöcke über alle 24 Schichten rund 24,9 Prozent, und die Feed-Forward-Netzwerke vereinen mit rund 49,7 Prozent fast die Hälfte der Parameter auf sich. Der größte Teil der Modellkapazität liegt also nicht in der Attention selbst, sondern in den Feed-Forward-Blöcken. Tabelle 1 fasst die zentralen Architekturwerte zusammen.

Tabelle 1: Zentrale Architekturwerte von Pythia-410M.

Eigenschaft	Wert
Architekturtyp	Decoder-Only (GPT-NeoX)
Vokabulargröße	50.304 Token
Hidden Size	1.024
Transformer-Schichten	24
Attention Heads je Schicht	16
Positionskodierung	Rotary Positional Embeddings (RoPE)

2.4 Datensätze

Für die verschiedenen Analysen dieser Arbeit werden drei sich ergänzende Datensätze verwendet.

Das **Hu-&-Liu-Opinion-Lexikon** liefert 2.254 Ein-Token-Wörter (865 positiv, 1.389 negativ), die vom Tokenizer von Pythia-410M jeweils als genau ein Token dargestellt werden. Das ursprüngliche Lexikon umfasst rund 6.800 sentimenttragende Wörter. Die Filterung erfolgt, indem jedes Wort mit führendem Leerzeichen tokenisiert und nur dann übernommen wird, wenn genau eine Token-ID erzeugt wird. Dadurch kann jedem Wort eindeutig ein Embedding-Vektor sowie ein einzelner Logit-Wert zugeordnet werden. Das Lexikon dient in dieser Arbeit zur Identifikation sentimentbezogener Tokens sowie zur Berechnung von Sentiment-Scores und Logit-Differenzen über die Transformer-Schichten hinweg [9].

Das **Sentiment Challenge Dataset** enthält 836 binär gelabelte Sätze, die gezielt sprachliche Herausforderungen wie Negation, Ironie, Idiome, Kompositionalität oder gemischtes Sentiment abdecken. Der Datensatz wird verwendet, um das Verhalten des Modells unter kontrollierten sentimentbezogenen Bedingungen zu untersuchen und dient als Grundlage für die qualitativen Analysen in Kapitel 3 [10].

Die **Counterfactually Augmented Data (CAD)** bestehen aus 1.707 Paaren positiver und negativer Texte, die sich nur in wenigen sentimenttragenden Wörtern unterscheiden. Da beide Varianten nahezu identischen Kontext besitzen, eignen sie sich besonders gut zur isolierten Untersuchung von Sentiment-Repräsentationen. In dieser Arbeit werden die CAD-Paare für die aggregierten Logit-Lens-, Attention-Head-, Linear-Probing- und Activation-Patching-Analysen verwendet [11].

3 Verhalten des Modells

Bevor die internen Mechanismen untersucht werden, lohnt sich ein einfacher Verhaltens-test. Was sagt das Modell überhaupt, wenn man es mit einem sentimentbezogenen Satz konfrontiert? Auf eine positiv formulierte Eingabe wie **The food was delicious and the**

`service was` antwortet Pythia-410M überwiegend mit positiven Fortsetzungen wie „excellent“ oder „great“, auf eine negative Eingabe entsprechend mit „terrible“ oder „awful“. Das Modell erkennt also grundsätzlich emotionalen Kontext.

Für eine systematische Prüfung wird der gefilterte Sentiment-Challenge-Datensatz (636 Sätze, 353 negativ, 283 positiv) verwendet. An jeden Satz wird ein Prompt-Suffix angehängt, und es wird geprüft, ob unter den zehn wahrscheinlichsten nächsten Token mehr positive oder negative Lexikonwörter sind. Mit dem Suffix `I feel very` erreicht das Modell eine Trefferquote von 75,2 Prozent, mit dem schwächeren Suffix `Sentiment:` nur 32,1 Prozent (Tabelle 2).

Tabelle 2: Konfusionsmatrix für den Suffix `I feel very` (Trefferquote 75,2 %).

wahr	vorhergesagt		
	positiv	negativ	kein Sentiment
positiv	259	24	0
negativ	134	219	0

Tabelle 3: Trefferquote ausgewählter Kategorien (Suffix `I feel very`).

Kategorie	n	Trefferquote (%)
positiv	272	86,4
negated	76	84,2
amplified	68	83,8
idioms	113	80,5
world-knowledge	72	62,5
sarcasm/irony	45	46,7

Tabelle 3 zeigt die Trefferquote nach Annotationskategorie für den besseren Suffix. Eindeutige Fälle wie reine positive oder negative Sätze, Verneinungen (negated) oder Verstärkungen (amplified) werden mit über 80 Prozent gut erkannt. Am schwierigsten ist die Kategorie Sarkasmus/Ironie mit nur 46,7 Prozent. Auffällig ist außerdem eine starke Neigung zur positiven Klasse. Beim schwächeren Suffix `Sentiment:` sagt das Modell in 395 von 636 Fällen „positiv“ voraus und liefert in 229 weiteren Fällen gar kein erkennbares Sentiment-Token. Diese niedrige Quote bei Sarkasmus passt zur Erwartung, dass das Modell Ironie nicht versteht.

4 Sentiment im Embedding-Raum

Dieses Kapitel prüft den beschreibenden Teil von Hypothese H1. Ist Sentiment schon vorhanden, bevor das Modell überhaupt eine Transformer-Schicht durchlaufen hat? Jedes Wort wird zunächst auf einen 1.024-dimensionalen Vektor abgebildet, das sogenannte Embedding.

Wenn Sentiment bereits hier erkennbar ist, muss das Modell es nicht erst mühsam aus dem Kontext erschließen.

Eine Hauptkomponentenanalyse reduziert diese 1.024 Dimensionen auf die zwei Achsen, die am meisten Streuung der 2.254 Hu-&-Liu-Wörter erfassen. Abbildung 1 zeigt, dass sich positive und negative Wörter auf diesen zwei Achsen zwar nicht vollständig trennen, aber bereits eine erkennbare Tendenz zeigen. Positive Wörter liegen im Histogramm eher links, negative eher rechts.

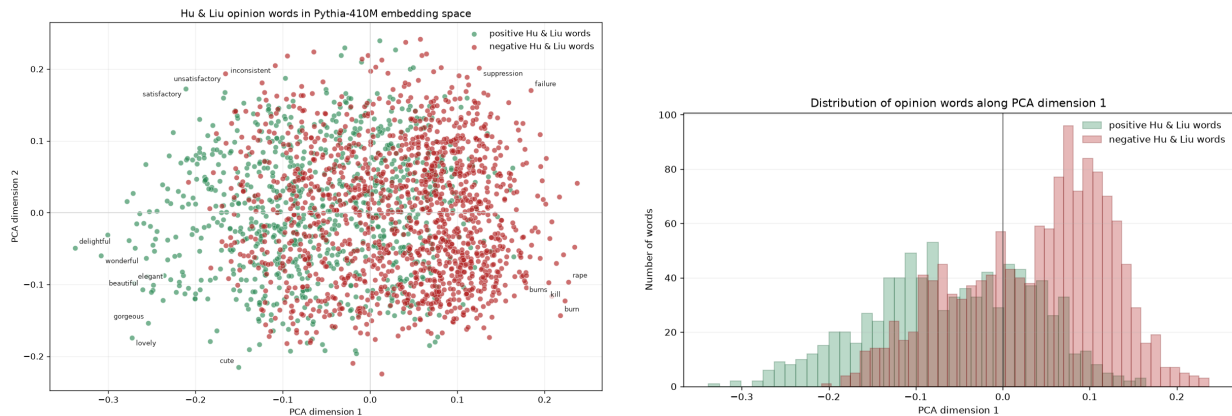


Abbildung 1: Hu-&-Liu-Wörter im Raum der ersten beiden Hauptkomponenten (links) und ihre Verteilung entlang der ersten Hauptkomponente (rechts).

Dass die Trennung auf nur zwei Achsen unvollständig bleibt, heißt nicht, dass Sentiment nicht vorhanden ist, sondern nur, dass die Information über viele der 1.024 Dimensionen verteilt ist. Auch die reine Vektorlänge (die L2-Norm) liefert kaum einen Hinweis. Sie liegt für positive und negative Wörter in jedem thematischen Feld nah beieinander (Abbildung 2), wenn auch mit einem leichten, über fast alle Felder konsistenten Muster, dass negative Wörter wie „sour“ oder „dangerous“ eine etwas größere Norm haben als ihre positiven Gegenstücke wie „delicious“ oder „safe“. Aussagekräftiger als die Länge ist die Richtung der Vektoren.

Ein ergänzender Blick auf die nächsten Nachbarn im Embedding-Raum, gemessen über die Cosine-Ähnlichkeit (den Winkel zwischen zwei Vektoren, unabhängig von ihrer Länge), bestätigt das Bild. Die einem Wort am nächsten liegenden anderen Wörter besitzen meist dieselbe Sentiment-Polarität, positive Wörter sind also überwiegend von positiven Nachbarn umgeben und negative von negativen. Sogar einfache Embedding-Arithmetik funktioniert in diese Richtung. Addiert man mehrere positive Wörter und zieht ein negatives ab, liegen die nächsten Nachbarn des Ergebnisvektors überwiegend im positiven Bereich, und umgekehrt.

Das zeigt sich noch deutlicher, wenn man gezielt die eine Richtung betrachtet, die aus der Differenz der Embeddings von „good“ und „bad“ gebildet wird. Projiziert man alle Sentiment-Wörter auf diese eine Achse, ergibt sich eine deutlich klarere Trennung. In allen zehn betrachteten thematischen Feldern, von „Food and taste“ über „Safety and trust“ bis

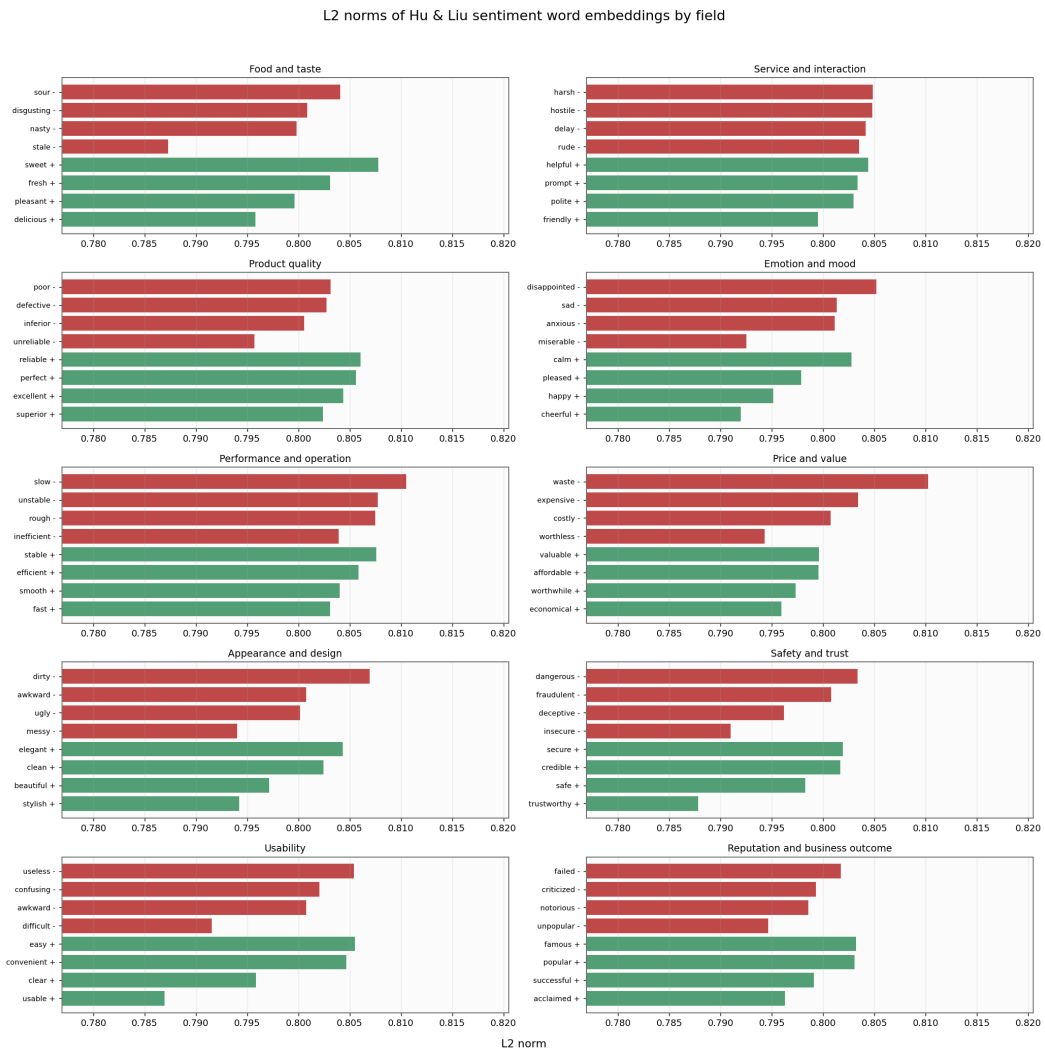


Abbildung 2: L2-Normen der Sentiment-Embeddings getrennt nach thematischen Feldern.

„Reputation and business outcome“, liegen positive Wörter fast durchgängig auf der positiven Seite dieser Achse und negative Wörter auf der negativen Seite, mit kaum Überlappung nahe der Mitte. Stärker konnotierte Wörter wie „excellent“ oder „dangerous“ liegen dabei weiter von der Mitte entfernt als schwächer konnotierte wie „pleasant“ oder „rude“. Die Richtung „good“ minus „bad“ verhält sich damit wie eine generelle Sentiment-Achse, die über verschiedenste Themenfelder hinweg funktioniert, obwohl sie nur aus einem einzigen Wortpaar gebildet wurde. Im Feld „Safety and trust“ etwa liegt „dangerous“ deutlich auf der negativen Seite und „trustworthy“ deutlich auf der positiven, während schwächer konnotierte Wörter wie „secure“ oder „insecure“ näher an der Mitte liegen. Die Achse bildet also nicht nur die Richtung, sondern auch die Stärke des Sentiments ab.

Um diese Beobachtung zu quantifizieren, wird ein lineares Probing durchgeführt. Eine logistische Regression, ein einfacher linearer Klassifikator, wird auf den Embeddings trainiert und soll positiv von negativ unterscheiden. Gelingt das gut, muss die Information bereits klar im Vektorraum vorliegen, ohne dass eine komplexe Berechnung nötig wäre. Die 2.254 Wörter werden dazu geschichtet im Verhältnis 80 zu 20 aufgeteilt (Tabelle 4).

Tabelle 4: Geschichtete Aufteilung der Hu-&-Liu-Wörter für das lineare Probing.

Teilmenge	positiv	negativ	gesamt
Training	692	1.111	1.803
Test	173	278	451
gesamt	865	1.389	2.254

Auf den Testdaten erreicht die Probe 97 Prozent Genauigkeit, bei gleichzeitig hohen Präzisions-, Recall- und F1-Werten für beide Klassen. Ordnet man jedes Testwort nach seiner vorhergesagten Wahrscheinlichkeit für positives Sentiment zwischen 0 und 1 an, liegt die überwiegende Mehrheit der Wörter klar auf einer der beiden Seiten der Entscheidungsgrenze von 0,5, nur eine kleine Gruppe liegt nahe an dieser Grenze und wird falsch klassifiziert. Die verbleibenden Fehler betreffen erkennbar mehrdeutige oder kontextabhängige Wörter. So werden „capricious“ und „tout“ positiv vorhergesagt, obwohl sie im Lexikon als negativ gelten, und „morality“ liegt fast exakt auf der Entscheidungsgrenze, passend dazu, dass eine moralische Bewertung ohne Kontext kein eindeutiges Sentiment trägt.

Damit lässt sich festhalten, dass der Embedding-Raum von Pythia-410M bereits eine linear weitgehend separierbare Sentiment-Struktur enthält, bevor überhaupt eine Transformer-Schicht durchlaufen wurde, sodass der beschreibende Teil von Hypothese H1 gestützt ist. Offen bleibt, ab welcher Schicht das Modell diese Information für seine tatsächliche Vorhersage nutzt; das untersucht das nächste Kapitel.

5 Schichtweise Entstehung mit dem Logit Lens

Dieses Kapitel untersucht, ab welcher Schicht und wie deutlich sich positive und negative Eingaben innerhalb des Modells unterscheiden, ein zentraler Baustein für den beschreibenden Teil von Hypothese H1. Am Ende des Modells wird der finale Zustand mit der Ausgabematrix auf den Vokabularraum projiziert, so entstehen die Logits, aus denen das nächste Token vorhergesagt wird. Der **Logit Lens** wendet genau dieselbe Ausgabematrix bereits auf frühere, eigentlich noch unfertige Zwischenzustände an [6]. So lässt sich für jede Schicht ablesen, welches Token das Modell zu diesem Zeitpunkt schon bevorzugen würde, ein Blick in den laufenden Denkprozess, wenn auch mit dem Vorbehalt, dass die Ausgabematrix nur für die letzte Schicht trainiert wurde und mittlere Schichten Information anders kodieren könnten.

Als Sentiment-Score dient die durchschnittliche Logit-Differenz zwischen allen positiven und allen negativen Hu-&-Liu-Wörtern je Schicht. An einem Beispielpaar (**The food was tasty/disgusting and I feel very**) liegt dieser Wert in den frühen Schichten nahe null. Positive und negative Eingaben sind für das Modell an dieser Stelle kaum zu unterscheiden. Ab den mittleren Schichten baut sich eine Trennung auf, die zwischen Schicht 17 und 23 ihre höchsten Werte erreicht (Abbildung 3). Der mit Abstand größte Einzelsprung liegt jedoch im letzten Schritt von Schicht 23 auf 24, wo sich der Effekt mehr als verdoppelt; die allerletzte Schicht scheint zusätzlich die konkrete Wortwahl zu optimieren.

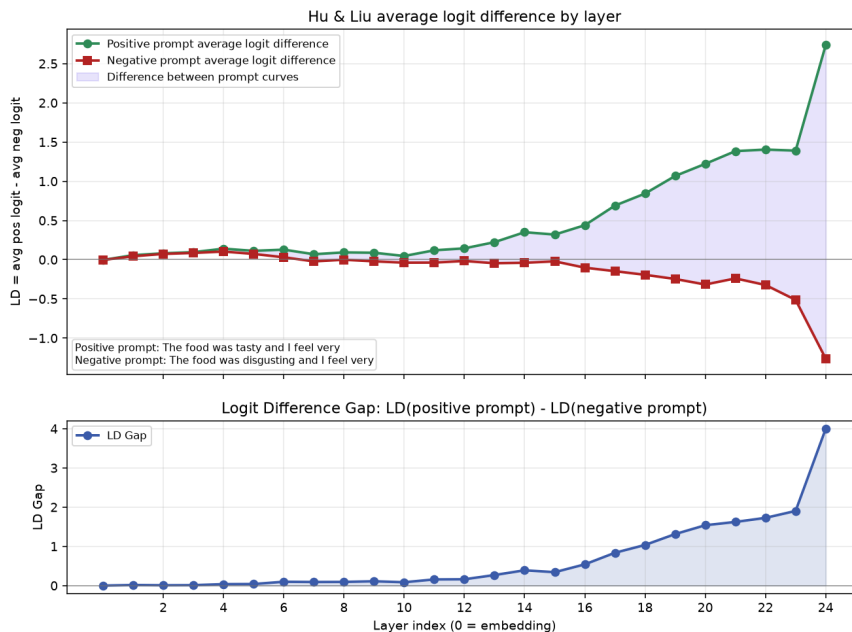


Abbildung 3: Logit-Differenz je Schicht für ein Beispiel-Prompt-Paar, oben die Differenz je Prompt, unten die Lücke zwischen positivem und negativem Prompt.

Betrachtet man statt der Differenz die absoluten Scores beider Klassen (Abbildung 4), zeigt sich ein überraschendes Zwischenmuster. Sowohl positive als auch negative Lexikon-

wörter sinken ab etwa Schicht 13 zunächst gemeinsam auf deutlich negative Werte ab, bevor in der letzten Schicht der Sprung ins Positive erfolgt. Das Modell verteilt die Logit-Masse zwischenzeitlich offenbar auf andere Token, statt durchgehend auf Sentiment-Wörter zu setzen; die eigentliche Trennung zwischen den Klassen bleibt davon unberührt, die Kurve ist aber kein einfacher, gleichmäßiger Anstieg.

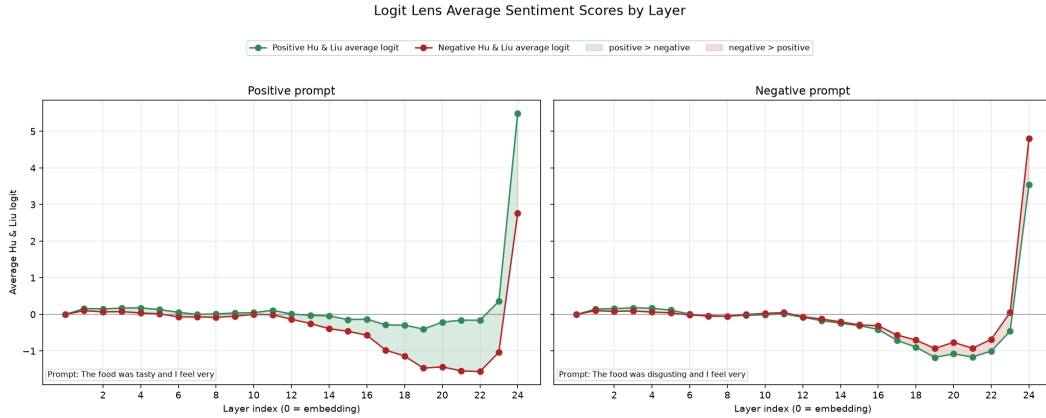


Abbildung 4: Absolute Sentiment-Scores je Schicht für ein positives und ein negatives Beispiel-Prompt.

Damit dieses Muster nicht nur an einem einzelnen, handverlesenen Beispiel gilt, wird es über alle 1.707 Paare des CAD-Datensatzes aggregiert (Abbildung 5). Das bestätigt den Befund systematisch. Der mittlere Abstand ist in den frühen Schichten nahe null, das Streuband reicht dort bis an die Null-Linie heran, sodass für viele Paare noch keine verlässliche Trennung besteht. Er wächst ab etwa Schicht 16 deutlich an und erreicht sein Maximum mit dem Sprung von Schicht 23 auf 24.

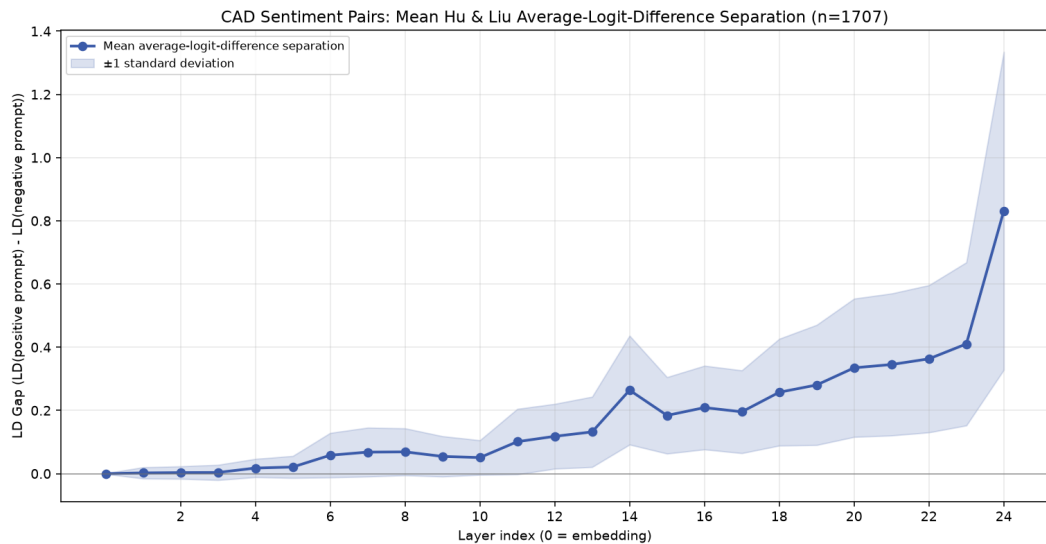


Abbildung 5: Über 1.707 CAD-Paare gemittelte Logit-Differenz je Schicht (Mittelwert ± 1 Standardabweichung).

Damit lässt sich festhalten, dass Sentiment nicht in einer einzelnen Schicht gespeichert,

sondern schrittweise über den Residual Stream aufgebaut wird, mit der deutlichsten Trennung in den oberen Schichten 17 bis 23 und dem größten Einzelschritt am Übergang zur letzten Schicht. Wichtig ist dabei der methodische Vorbehalt aus der Einleitung dieses Kapitels. Da die verwendete Ausgabematrix nur für die letzte Schicht optimiert wurde, könnte der Logit Lens echte Sentiment-Information in mittleren Schichten unterschätzen, wenn diese dort noch in einer anderen, dem Modell eigenen Kodierung vorliegt und erst später in die vom Lens lesbare Form überführt wird. Dieser Befund macht den Bereich um Schicht 17 bis 23 trotzdem zum naheliegenden Kandidaten für die kausale Überprüfung in Kapitel 7; ob sich diese Erwartung bestätigt, zeigt sich dort.

6 Informationsfluss über die Attention

Dieses Kapitel untersucht, welche Attention Heads ihre Aufmerksamkeit auf sentimenttragende Wörter richten, und damit als Kandidaten für die kausale Verarbeitung in Hypothese H1 in Frage kommen. Jeder Kopf berechnet für jede Position eine gewichtete Summe über alle vorherigen Token; ein Gewicht nahe 1 bedeutet, dass eine Position fast ihre gesamte Aufmerksamkeit auf ein einzelnes anderes Token legt. Man kann sich das als eine Frage-und-Antwort-Beziehung vorstellen. Jedes Token stellt über seine Query eine Art Suchanfrage, und jedes andere Token bietet über seinen Key eine passende oder unpassende Antwort an. Je besser Anfrage und Antwort zusammenpassen, desto höher das Attention-Gewicht und desto mehr Information fließt von diesem Token. Bei Pythia-410M gibt es 24 Schichten mit je 16 solchen Köpfen, also 384 Stellen, an denen Information auf diese Weise fließen kann.

An einem Beispielsatz wird untersucht, welche Köpfe besonders stark auf das Wort „delicious“ achten. Die stärkste gemessene Aufmerksamkeit liegt in Schicht 6, Kopf 3, mit einem Gewicht von 0,835 (Abbildung 6); weitere auffällige Köpfe liegen in den Schichten 0 bis 12. Die stärksten Muster konzentrieren sich also auf frühe bis mittlere Schichten, genau dort, wo der Logit Lens noch kaum eine Sentiment-Trennung zeigt.

Ein einzelner Beispielsatz kann jedoch täuschen. Aggregiert über alle Prompts des CAD-Datensatzes relativiert sich dieser Befund deutlich. Der höchste gemittelte Aufmerksamkeitswert auf sentimenttragende Wörter liegt bei nur 0,019 (Abbildung 7), fast zwei Größenordnungen niedriger als am Einzelbeispiel, und Schicht 6/Kopf 3 fällt im Aggregat nicht besonders auf. Systematisch über den Datensatz hinweg ist die Aufmerksamkeit auf einzelne Sentiment-Wörter also schwach und diffus über viele Köpfe verteilt, statt auf wenige dominante Köpfe konzentriert zu sein.

Ergänzend wurde nach Induction Heads gesucht, also Köpfen, die Muster der Form „A B ... A“ zu „B“ vervollständigen und als bekannter Mechanismus zur Mustererkennung gelten. Die höchsten Induction-Scores (zwischen 0,38 und 0,44) liegen in den frühen Schichten 1, 3, 4 und 5, also in einem ähnlichen Bereich wie die stärkste sentimentbezogene Aufmerksamkeit, aber nicht bei denselben Köpfen.

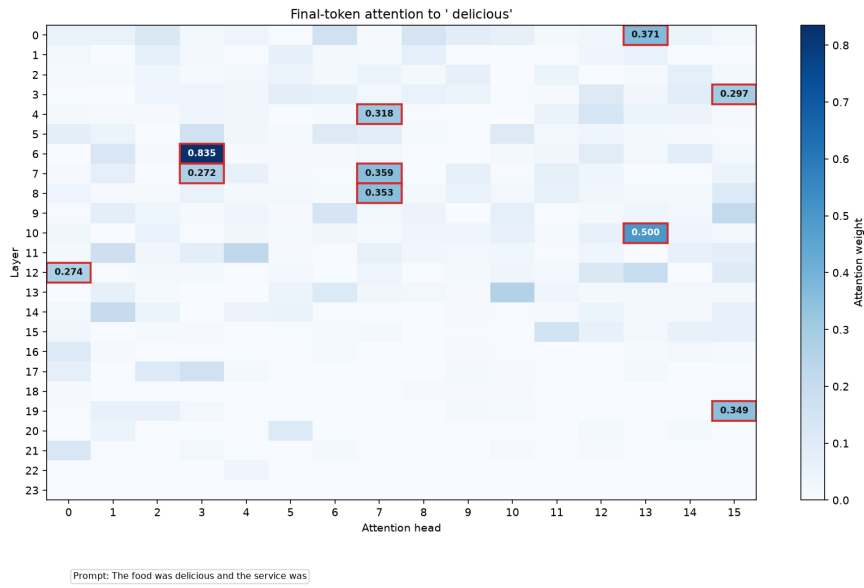


Abbildung 6: Aufmerksamkeit der Köpfe auf das sentimenttragende Token „delicious“ an einem Beispielsatz.

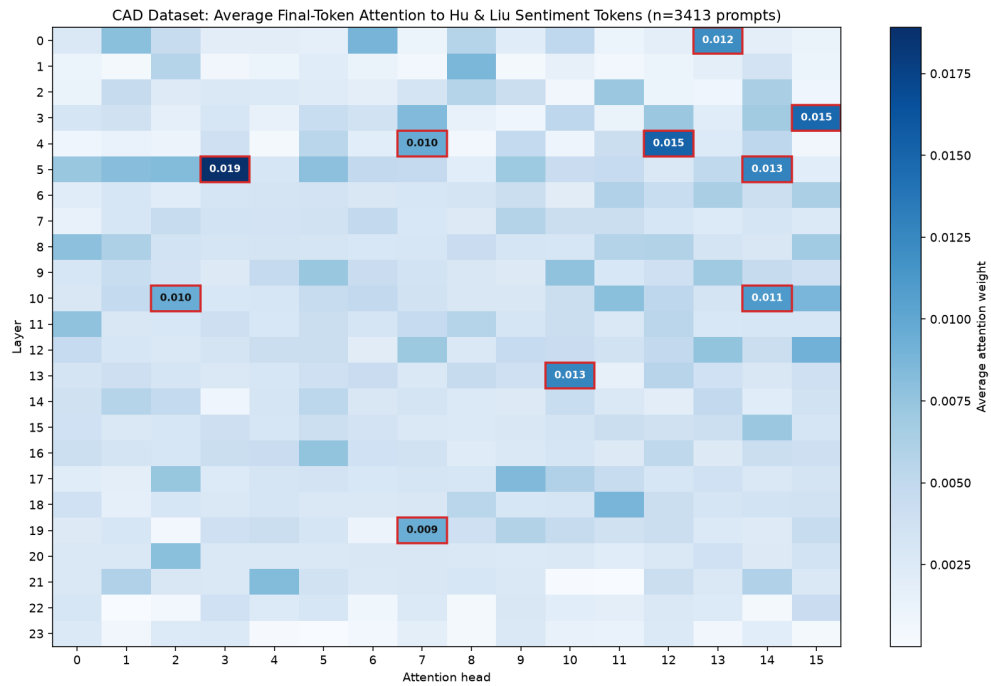


Abbildung 7: Durchschnittliche Aufmerksamkeit je Schicht und Kopf auf sentimenttragende Wörter über den CAD-Datensatz (n=3413 Prompts).

Damit lässt sich nur mit Vorbehalt festhalten, dass es Köpfe gibt, die auf sentimenttragende Wörter fokussieren, aber dieser Effekt ist im Aggregat schwach. Vor allem zeigt hohe Aufmerksamkeit nur, dass ein Kopf Information von einem Token *bezieht*, nicht, dass diese Information für die Vorhersage tatsächlich *gebraucht* wird. Attention beschreibt damit nur den Informationsfluss, nicht die Kausalität. Ob ein Kopf oder eine Schicht wirklich notwendig ist, kann nur ein aktiver Eingriff zeigen; das leistet das Activation Patching im folgenden Kapitel.

7 Kausale Überprüfung mit Activation Patching

Alle bisherigen Verfahren liefern Korrelationen. Sie zeigen, dass eine Information vorhanden ist oder genutzt zu werden scheint, aber nicht, dass eine bestimmte Komponente für die Vorhersage tatsächlich notwendig ist. Diesen Nachweis liefert das **Activation Patching**, das in der mechanistischen Interpretierbarkeit als kausaler Goldstandard gilt [7], [8]. Das Prinzip ist ein kontrolliertes Experiment, bei dem ein *sauberer* und ein *korruptierter* Durchlauf verwendet werden, die sich nur im sentimenttragenden Wort unterscheiden. Im korruptierten Durchlauf wird dann gezielt die Aktivierung einer einzelnen Schicht an einer einzelnen Token-Position durch die zuvor gespeicherte saubere Version ersetzt. Verändert sich die Vorhersage dadurch wieder in Richtung des sauberen Ergebnisses, enthält genau diese Schicht und Position die entscheidende Information. Normalisiert bedeutet ein Effekt von 0 keine Wiederherstellung, ein Effekt von 1 eine vollständige Wiederherstellung der sauberen Vorhersage. Technisch läuft das über PyTorch Forward Hooks. Zunächst werden alle sauberen Aktivierungen zwischengespeichert, danach wird systematisch über jede Kombination aus Schicht und Position gepatcht und der jeweilige Effekt gemessen, und zuletzt werden die Ergebnisse als Heatmap dargestellt und die einflussreichsten Punkte herausgefiltert.

Als Prompt-Paar dient erneut das bereits aus den vorigen Kapiteln bekannte Beispiel `The food was delicious/disgusting and I felt very` mit Zielwort „satisfied“, derselben Formulierung, an der schon der Logit Lens und die Attention-Analyse untersucht wurden, sodass sich die drei Methoden direkt miteinander vergleichen lassen. Im sauberen Durchlauf erreicht dieses Wort einen Logit von 15,78 (Wahrscheinlichkeit 10,2 Prozent), im korruptierten Durchlauf nur 9,21 (Wahrscheinlichkeit unter 0,1 Prozent). Aus der Erwartung der vorigen Kapitel heraus sollte der stärkste kausale Effekt in den oberen Schichten 17 bis 23 liegen, dort, wo Logit Lens und Attention die auffälligsten Muster zeigten.

Das tatsächliche Ergebnis überrascht. Neun der zehn stärksten Punkte liegen exakt an Position 3, dem Wort „delicious“, verteilt über die frühen Schichten 0 bis 8 (Tabelle 5, Abbildung 8). Die kausal entscheidende Information sitzt also nicht in den oberen, sondern bereits in den allerfrühesten Schichten an der Token-Repräsentation selbst, passend zur linearen Trennbarkeit der Embeddings aus Kapitel 4, aber im Widerspruch zur Erwartung aus Logit Lens und Attention.

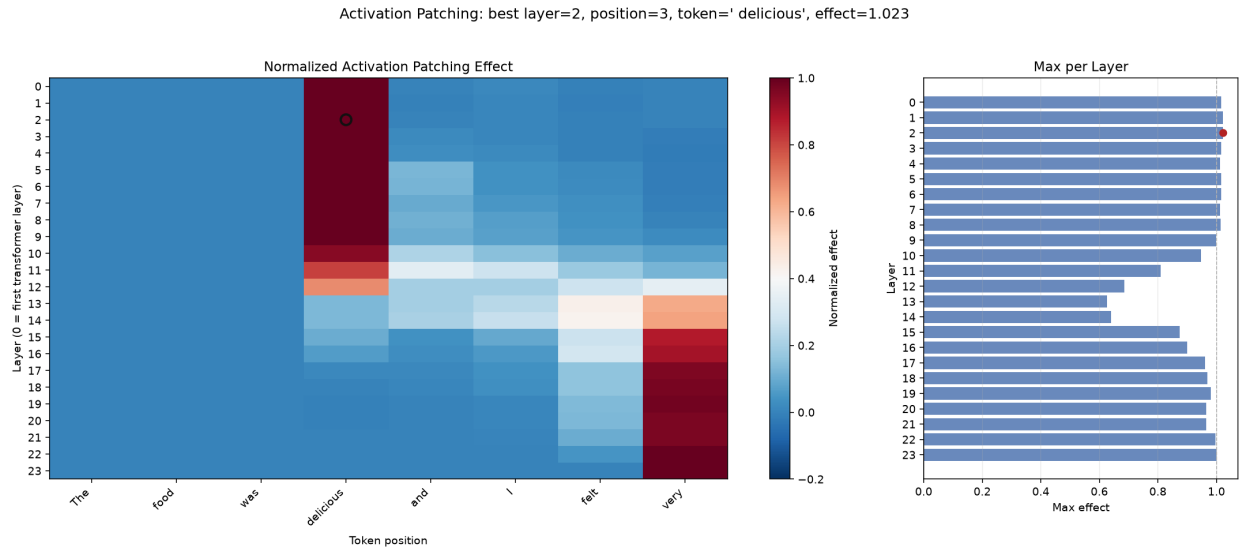


Abbildung 8: Normalisierter Activation-Patching-Effekt über alle Schichten und Token-Positionen.

Tabelle 5: Die fünf einflussreichsten Patching-Punkte.

Rang	Schicht	Position	Token	Effekt
1	2	3	delicious	1,023
2	1	3	delicious	1,021
3	0	3	delicious	1,017
4	3	3	delicious	1,016
5	6	3	delicious	1,016

Ergänzend wird auf Ebene einzelner Köpfe gepatcht. Statt der gesamten Schicht-Aktivierung wird nur der Anteil eines einzelnen Attention-Kopfes an Position 3 ersetzt (Abbildung 9). Der stärkste Einzelkopf liegt in Schicht 5, Kopf 12, mit einem Effekt von lediglich 0,019, rund zwei Größenordnungen schwächer als der Effekt auf Schichtebene. Kein einzelner Kopf erklärt den beobachteten Effekt auch nur annähernd, und insbesondere taucht der zuvor auffällige Kopf aus Kapitel 6 (Schicht 6, Kopf 3) hier nicht auf.

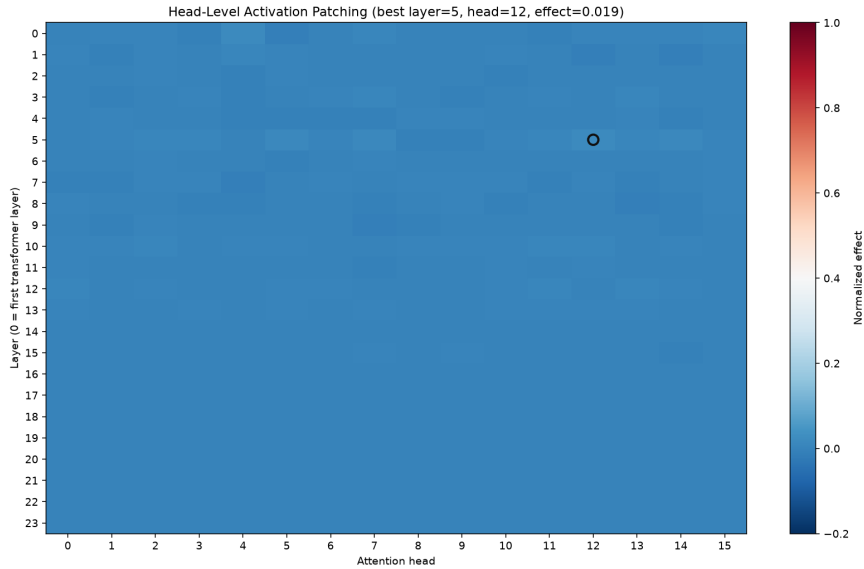


Abbildung 9: Patching auf Ebene einzelner Attention Heads an Position 3 (Token „delicious“).

Für dieses eine Prompt-Paar bestätigt sich damit der kausale Teil von Hypothese H1 vollständig, denn ein Eingriff am Stimmungswort stellt die Vorhersage wieder her. Hypothese H2 bestätigt sich nur teilweise. Die Position ist scharf lokalisiert, die Schicht dagegen nicht, sondern verteilt sich über ein Band aus neun frühen Schichten statt auf einen einzelnen Punkt. Da nur ein Beispiel geprüft wurde, ist offen, ob sich das Muster über den gesamten CAD-Datensatz verallgemeinern lässt.

8 Diskussion und Fazit

8.1 Zusammenfassung der Befunde

Sentiment ist bereits im Embedding-Raum strukturiert und zu 97 Prozent linear trennbar. Über die Schichten hinweg baut sich die Trennung schrittweise auf und erreicht ihr Maximum in den oberen Schichten 17 bis 23, mit dem größten Einzelsprung im letzten Schritt. Bei der Attention fokussieren Köpfe am Einzelbeispiel stark auf das Stimmungswort, im Aggregat über den CAD-Datensatz ist dieser Effekt jedoch schwach und diffus.

8.2 Hypothesen-Verdict

H1 (klare Stimmungswörter)

Bestätigt. Die Polarität ist früh linear ablesbar, und das Activation Patching zeigt für das geprüfte Beispiel, dass ein Eingriff am Stimmungswort die Vorhersage wiederherstellt.

H2 (Lokalisierbarkeit)

Teilweise bestätigt. Die Token-Position ist scharf lokalisierbar, die Schicht jedoch nicht, denn der Effekt verteilt sich über ein Band früher Schichten statt auf einen einzelnen Punkt. Zudem liegt der kausal wichtige Ort fundamental anders (sehr früh, token-nah) als die korrelativ aus Logit Lens und Attention-Mechanismus vermuteten oberen Schichten und Köpfe.

H3 (Ironie und Sarkasmus)

Nicht abschließend geklärt. Die niedrige Trefferquote bei Sarkasmus/Ironie im Verhaltenstest passt zur Hypothese, ist aber durch die starke Negativ-Schiefelage dieser Kategorie und den allgemeinen Positiv-Bias des Modells konfundiert. Eine schichtweise lineare Probe auf Ironie-Beispielen wurde nicht durchgeführt.

8.3 Korrelation versus Kausalität

Ein wichtiges Muster zieht sich durch die ganze Arbeit. Korrelative Verfahren und der kausale Test zeigen nicht an derselben Stelle ihre stärksten Effekte. Logit Lens und Attention-Mechanismus-Analyse legen die oberen Schichten beziehungsweise einen bestimmten Kopf in frühen bis mittleren Schichten als wichtig nahe. Das Activation Patching findet die kausal entscheidende Information jedoch schon an der Token-Repräsentation selbst, in den allerersten Schichten, und keinem einzelnen Kopf zuordenbar. Hohes Attention-Gewicht und eine deutliche Logit-Trennung sind demnach nützliche Hinweise, aber kein Ersatz für einen tatsächlichen Eingriff, ein Befund, der die Methodik dieser Arbeit selbst bestätigt, denn erst die Kombination aus mehreren Werkzeugen deckt solche Diskrepanzen auf. Hätte die Arbeit beim Logit Lens oder bei der Attention-Analyse aufgehört, wäre die naheliegende, aber falsche Schlussfolgerung gewesen, dass die oberen Schichten und der auffälligste Kopf kausal verantwortlich sind.

8.4 Grenzen

Der Logit Lens nutzt die für die letzte Schicht trainierte Ausgabematrix und kann mittlere Schichten daher unterschätzen. Die lexikonbasierte Bewertung übersieht Sentiment außerhalb des Lexikons, und der Ein-Token-Filter schränkt die betrachtete Wortmenge zusätzlich ein. Activation Patching wurde bislang nur für ein Prompt-Paar gerechnet, nicht über den gesamten CAD-Datensatz aggregiert; alle Befunde beziehen sich zudem auf ein einzelnes 410-Millionen-Parameter-Modell, weshalb sie sich nicht ohne Weiteres auf größere Sprach-

modelle übertragen lassen.

8.5 Fazit

Pythia-410M repräsentiert Sentiment bereits im Embedding, baut die Trennung über die Schichten weiter auf und lässt sich für ein Beispiel auch kausal auf eine frühe, token-nahe Stelle zurückführen, nicht auf die anhand von Logit Lens und Attention vermuteten oberen Schichten und Köpfe. H1 ist damit bestätigt, H2 nur teilweise, H3 bleibt offen. Der nächste und naheliegendste Schritt ist, das Activation Patching über mehrere oder alle 1.707 Paare des CAD-Datensatzes zu aggregieren, so wie es für Logit Lens und Attention bereits geschehen ist, um zu prüfen, ob sich die frühe, token-nahe Lokalisierung verallgemeinern lässt. Darüber hinaus bieten sich eine Erweiterung auf größere Modelle der Pythia-Reihe und eine gezielte schichtweise Prüfung von Hypothese H3 anhand von Ironie-Beispielen als weiterführende Arbeiten an.

Literaturverzeichnis

- [1] A. Vaswani u. a., “Attention Is All You Need,” in *Advances in Neural Information Processing Systems*, 2017.
- [2] A. Radford, K. Narasimhan, T. Salimans und I. Sutskever, “Improving Language Understanding by Generative Pre-Training,” 2018.
- [3] N. Nanda, *A Comprehensive Mechanistic Interpretability Explainer and Glossary*, <https://www.neelnanda.io/mechanistic-interpretability/glossary>, 2023.
- [4] N. Elhage u. a., “A Mathematical Framework for Transformer Circuits,” *Transformer Circuits Thread*, 2021.
- [5] S. Biderman u. a., “Pythia: A Suite for Analyzing Large Language Models Across Training and Scaling,” in *Proceedings of the 40th International Conference on Machine Learning*, 2023.
- [6] nostalgebraist, *Interpreting GPT: The Logit Lens*, LessWrong, <https://www.lesswrong.com/posts/AcKRB8wDpdaN6v6ru/interpreting-gpt-the-logit-lens>, 2020.
- [7] K. Meng, D. Bau, A. Andonian und Y. Belinkov, “Locating and Editing Factual Associations in GPT,” in *Advances in Neural Information Processing Systems*, 2022.
- [8] N. Goldowsky-Dill, C. MacLeod, L. Sato und A. Arora, “Localizing Model Behavior with Path Patching,” *arXiv preprint arXiv:2304.05969*, 2023.
- [9] M. Hu und B. Liu, “Mining and Summarizing Customer Reviews,” in *Proceedings of the Tenth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2004, S. 168–177.
- [10] J. Barnes, L. Øvrelid und E. Velldal, “Sentiment Analysis Is Not Solved! Assessing and Probing Sentiment Classification,” in *Proceedings of the 2019 ACL Workshop BlackboxNLP: Analyzing and Interpreting Neural Networks for NLP*, Florence, Italy: Association for Computational Linguistics, 2019, S. 12–23.
- [11] D. Kaushik, E. Hovy und Z. C. Lipton, “Learning the Difference that Makes a Difference with Counterfactually-Augmented Data,” in *International Conference on Learning Representations (ICLR)*, 2020.